

# 0 摘要

---

PANCAKE是第一个以较小带宽开销保护键值存储免受访问模式泄漏攻击的系统。它使用频率平滑方法，将明文访问转化为加密数据存储的均匀分布访问，并在新的安全模型中证明了其对被动持久对手的防护能力。

## 1 介绍

---

### 问题

---

许多组织为了便于管理和扩展，已从内部部署转向云托管数据存储，并从服务器连接转向分散存储。尽管这带来了好处，但曾经在受信任域中的数据访问现在可能对不受信任的实体可见。

一系列研究表明，即使数据被加密，观察到的数据访问模式仍可被利用，通过频率分析等访问模式攻击获取敏感信息。这些攻击只需被动持久对手，即观察访问模式而不主动执行访问的对手。现有的针对访问模式攻击的技术，如遗忘RAM，面向更强的安全模型，需对手主动访问，但这些技术存在显著性能开销，因此在大多数场景下不切实际。**因此，如何构建高性能的数据存储以抵御持久被动对手的访问模式攻击仍未解决。**

### 作者的三个核心贡献

---

1. 介绍了一个在加密数据存储设置中捕获被动持久敌手的安全模型  
安全的定义：对手无法分辨来自对随机比特串和均匀分布的访问序列的副本。我们说这是ROR-CDDA安全的。  
ROR-CDDA: real-or-random indistinguishability under chosen dynamic-distribution attack
2. 频率平滑  
提供针对被动持久对手的访问模式攻击的安全性。  
关键：对被动持久对手来说，从一种提供“不确定性”的分布中选择数据访问请求  
频率平滑结合了四种技术：选择性复制、虚假访问、查询批处理和动态自适应。
3. 端到端系统——PANCAKE的设计、实施和评估

PANCAKE使用了一种加密系统代理模型，这个代理充当在客户端和数据存储的中

介。PANCAKE使用此代理来维护时变访问分布的估计（基于来自客户端的传入请求），并通过使用伪随机函数作为密钥和对值进行身份验证加密来安全地执行读/写查询。

## 2 PANCAKE安全模型

---

### 系统模型

---

PANCAKE在键值(KV)存储中的应用，支持多个客户端的单键获取、放置和删除操作，同时适用于其他数据存储。它采用加密代理架构，多个客户端通过受信任的代理路由查询，代理管理查询执行并发送至存储服务。安全模型同样适用于单客户端环境，假设通信信道加密(如TLS)，但存储服务仍可见请求。代理通过伪随机函数加密KV对，确保值大小一致以避免长度泄漏，所需密钥存储在代理中，且代理可以通过请求 $F(k)$ 来执行密钥操作。

### 安全模型

---

介绍：被动持久对手：观察所有（加密）访问，但不主动执行自己的访问

我们将诚实的客户端请求建模为从密钥上的分布 $\pi$ 中采样的查询：对于每个密钥 $k$ ，对该密钥进行查询（get、put或delete）的概率表示为 $\pi(k)$ 。

假设：敌手没有密钥，但是可以观察加密请求和存储服务器的回应；敌手不会对信息做任何修改。敌手知道分布 $\pi$ ，但随机抽取是隐藏的。**如果对手能够推断出有关访问的明文KV对的结果序列的任何信息，则获胜。**

### 访问模式攻击

---

在作者的安全模型中，敌手能利用到：

1. 推断关键特征 (?)
2. 识别何时访问特定密钥
3. 检测并识别关键流行度随时间的变化

作者的目标：防止这种访问模式攻击。

### 相关工作

---

## 1、遗忘RAM (ORAM)

ORAM能为访问模式攻击提供保护，即使攻击者可以注入自己的查询。

ORAM开销的强下限：对于具有 $n$ 个键值对的数据存储，

1. 使用恒定的代理存储，但会产生 $\Omega(\log n)$ 带宽开销；
2. 必须在代理服务器上使用 $\Theta(n)$ 存储，并产生恒定的带宽开销。

对于庞大的数据来说很低效。

在较大规模下， $\Omega(\log n)$ 带宽开销会显著降低吞吐量。虽然最新的ORAM设计理论上实现了恒定带宽开销，但隐藏了较大的常数。一些研究通过仅支持写入来实现恒定带宽和效率，但对于需要完整ORAM功能的应用，ORAM开销可能不可接受。

## 2、快照攻击 (snapshot attacks)

对手不会持续观察查询，只会获得加密数据存储的一次性副本（快照）

快照模型对于现有的存储系统来说是不切实际的

## 3、假查询

# 3 PANCAKE概述

PANCAKE的核心——频率平滑

## 频率平滑

PANCAKE的设计利用了所有客户端通过代理路由查询的特点；因此，代理可以学习关于明文访问频率的信息。

**PANCAKE使用 $\pi$ 的估计来执行频率平滑**

关键：如何有效地将根据 $\pi$ 在（明文）关键词分布的访问转换为加密密钥上的均匀分布

PANCAKE通过三个技术来实现之：选择性复制、假查询和批处理。

选择性复制：创建与访问可能性成比例的密钥 $k$ 的多个副本，当访问一个密钥时，会选取这个密钥的一个副本。

然而，PANCAKE创建R(k)个K的副本，只需要保证  $\pi(k)/R(k) \leq 1/n$ ，其中n是数据存储中的项目数。我们将  $\alpha = 1/n$  称为复制阈值。副本的个数记作n'，虽然但是，总数会  $\leq 2n$ 。有时攻击者可能会知道一些关于 n' 的分布，我们加上了假密钥D的 2n-n' 个副本。因此，副本的总数总是2n，与π无关。

然后PANCAKE使用相同比例的假查询与真实查询混合在一起，以确保访问的均匀分布。

解释—下文章第三小节中频率平滑关于服务客户端查询的运作方式

在文章第三小节中，频率平滑（frequency smoothing）机制通过一系列步骤来服务客户端的查询。具体运作方式如下：

1. 查询选择：当接收到客户端的真实查询时，PANCAKE会执行一个过程，从真实和假查询的分布中随机选择一系列的访问请求。这个过程确保了真实查询以一定的机率被包含在所选中的请求序列中。

2. 请求插入：在选定的请求中，PANCAKE会将客户端的实际请求插入到随机选择的请求中。这样做的目的是为了在保持查询隐私的同时，平滑访问模式，防止潜在的攻击者从中获取有价值的信息。

3. 请求排队：在某些情况下，可能会出现客户端的请求无法立即被服务。这时，PANCAKE会将该请求放入一个队列中，等待下一个客户端请求到达时再进行处理。

4. 批量处理：为了提高效率，PANCAKE采用了批量处理（batching）的方法，例如，设置一个批次大小B（如B=3），确保在处理过程中能够及时交付客户端的请求，同时保证每个访问请求的成功概率是均等的。

5. 性能保证：通过上述机制，PANCAKE能够确保客户端请求的及时交付，并且有效防止查询模式被攻击者识别。文中提到，选择合适的批量大小和假查询与真实查询的比例是实现高效和安全的关键[10][11]。

总的来说，频率平滑机制通过随机选择、插入真实请求、排队和批量处理等方式，有效地维护了数据访问模式的安全性和隐私性，同时优化了系统性能。

## 动态查询分布

为了确保PANCAKE在访问分布变化时仍能保持安全性和性能，我们采用了一种高效算法，动态更新虚假查询的概率和关键词的副本分配。PANCAKE始终保持2n个副本数量，无论分布如何。当分布变化时，每个需要减少副本的关键词必须有另一个关键词增加副本。因此，处理分布变化只需重新分配这些关键词对的副本。PANCAKE采用专门的副本交换协议，能够在同时处理客户端请求的情况下高效调整副本分配。主要挑战在于，必须用旧副本服务请求，而不能使用新分配的副本，直到新副本正确传播值为止。我们展示了可以临时降低真实查询与虚假查询的比例，结合临时缓存值，保持访问存储的均匀分布，从而保证安全性。

## 4 PANCAKE设计：静态分布

### 4.1 数据存储

#### PANCAKE代理

主要功能：初始化数据存储，实行频率平滑，并且代表客户端执行查询

代理维护了多个结构来实现其功能：

- **观察的查询分布**( $\hat{\pi}$ )：代理通过密钥访问的直方图维护访问概率，该“观察到的”分布估计潜在分布并用于检测随时间的变化。
- **虚假查询分布**( $\pi_f$ )：代理还为每个单独的密钥维护一个虚假的访问概率。
- **密钥→副本计数**：使用选择复制机制创建KV对。
- **更新缓存**：一种存储映射  $k \rightarrow (v, UpdateMap)$  的数据结构，其中UpdateMap是一个长度为  $R(k)$  的位图，表示k的特定副本是否已更新。
- **查询队列**：存储未完成的客户端查询。

PANCAKE代理的存储要求：密钥的概率存储为float，只需要8字节的存储空间，因此存储真实和虚假分布只需要整个数据集大小的一小部分。另一方面，UpdateCache的大小取决于查询分布和写入速率。

PANCAKE代理的可拓展性：PANCAKE代理实现了多核高效扩展。对于多核实现，前四个数据结构由所有PANCAKE代理核心共享，而每个核心都维护自己的查询队列（用于“分配”给该核心的查询）。

## 4.2 频率平滑

它将明文数据库  $KV = \{(k_i, v_i)\}$  上的真实查询转换为加密数据库  $KV'$  上的固定访问次数（如“批量大小”为B）。通过创建多个KV对的副本并生成虚假查询，确保每个B访问都是  $KV'$  中项目的均匀随机选择。

### 初始化数据存储

我们将加密后的关键词称为标签

1. PANCAKE将KV密钥上按 $\pi$ 分布的访问转换为KV'加密密钥上的统一访问序列。为了区分明文密钥和加密密钥，我们将后者称为标签PANCAKE使用 $\pi$ 的 $\hat{\pi}$ 估计。 $\hat{\pi}$ 可以被指定为均匀分布，或者从性能或其他日志中学习进行热启动。

根据 $\hat{\pi}$ 中关键词的访问频率，使用选择性复制添加副本以生成  $KV'$

设置一个阈值 $\alpha$ ，然后对每个  $(k, v) \in KV$  生成  $R(k, \hat{\pi}, \alpha) = \lceil \hat{\pi} / \alpha \rceil$  个副本：  
 $((k, j), v)$ ，其中  $j \in [1, R(k, \hat{\pi}, \alpha)]$

然后，通过伪随机函数F（例如HMAC）应用于副本标识符来生成标签  $F(k, i)$ ，从而保护每个副本  $(k, i)$ 。我们对该值应用认证加密E。那么最终的  $KV'$  对的形式是

$$KV' = \{F(k_i, j), E(v_i)\}, 1 \leq i \leq n, 1 \leq j \leq R(k_i)$$

把 $KV'$ 的总数记为 $n'$ ，别管为什么， $n' \leq n + 1/\alpha$

## 2. 计算副本上的伪分布 $\pi_f$

采用一个01之间的常数" $\delta$ "，然后处理 $\pi_f$ ，使得访问任何副本 $(k, j)$ 的概率 $p(k, j)$ 为：

(1) 等于 $1/n'$

(2) 真实访问副本和执行虚假访问的概率的凸组合

那么就是说

$$p(k, j) = \delta \cdot \frac{\hat{\pi}(k)}{R(k)} + (1 - \delta) \cdot \pi_f(k, j) = \frac{1}{n'}$$

特别的，对任意的 $k$ ，需要满足，否则，可能无法为某个关键词 $k$ 分配有效（非负）概率来满足上述方程，当时，上述方程的结果始终有效。

由上述方程显然可以发现， $\delta$ 和真查询的比例是相关的，当 $\alpha$ 增加时，大部分的查询会变成假查询；当 $\alpha$ 减小时，又会增加 $KV'$ 的数量。

因此我们设 $\alpha = 1/n$ 使得 $n' \leq 2n$  i.e.  $\#KV' \leq 2\#KV$  并且  $\delta \geq 1/2$ ，就是说大部分的查询是真查询。

## 假副本

易知对于不同的分布，会有不同数量的副本（不会超过 $2n$ ），但这会泄露不同的信息。所以PANCAKE系统加上了假副本，始终维持副本的数量在 $2n$

假副本是KV对 $(F(D, j), E(D))$ ，其中 $j=1, \dots, 2n-n'$  ( $n'$ 是 $\pi$ 的“实”副本的数量)

注意：dummy replicas不存在于原始的关键字中，只有通过虚假访问才能访问到他。



notice: D, R don't exist in original keys sets.  
 so  $\hat{\pi}(D) = 0$  and from (1) we know

$$\frac{1}{n'} = (1 - \delta) \frac{\pi_f(D)}{\pi_f(D)} = \alpha / (n'\alpha - 1) = \alpha / (2n\alpha - 1)$$

since we let  $n' = 2n$

$$\frac{1}{2n\alpha} = \frac{\alpha}{2n\alpha - 1} \quad \delta = 1 / 2n\alpha = 1/2 \quad \text{for } \alpha = \frac{1}{n}$$

$$\frac{n'\alpha - 1}{n'\alpha} \pi_f(D) = \frac{1}{n'}$$

## 执行查询

$x \xrightarrow{\$} X$ : 元素  $x$  从集合  $x$  中随机均匀采样

为了提高客户端的真实访问被立即处理的概率, PANCAKE 实际上为每个客户端请求向 KV 发送了一小批访问。

For each  $q_{type} = 1$  (real),

send a value from query queue

if Queue is Empty

client simulate real query

For each  $q_{type} = 0$  (fake)

client sample according to  $\pi$

The resulting batch of replicas have PRF applied before sent to server.

Query execution

Batch  $(k)$

$\hat{j} \in \{1, \dots, R(k)\}$

Add to Queue  $(k, \hat{j})$

For  $i = 0$  to  $B$

$q_{type} \in \{0, 1\}$

if  $q_{type} = 0$

$(k_i, j_i) \in \pi$

Else:

If Queue Not Empty:

$(k_i, j_i) \in \text{DeQueue}()$

Else:

$k_i \in \pi$

$j_i \in \{1, \dots, R(k)\}$

$d \in \{F(k_1, j_1), \dots, F(k_B, j_B)\}$

请注意，在PANCAKE代理中完成的批处理不需要将批处理中的所有查询发送到同一个分片/服务器；批处理完全独立于服务器上使用的分片机制，批处理中的查询被独立转发到各个分片。检索到相关值后，PANCAKE对客户端请求的值进行解密并返回。

PANCAKE只为每个客户端请求发送一个批处理。如果代理发送批处理直到查询队列为空，则有关客户端访问哪些密钥的频率信息将泄露。例如，如果使用 $B=1$ 并一直提交，直到队列为空，那么对KV的最终访问必须是客户端请求。因此，如果有必要，PANCAKE会将查询的处理推迟到稍后的批处理，从而增加延迟。这种延迟可以接受。事实上我们可以根据负载变化 $B$ 。



## 支持更新

PANCAKE通过借用ORAM文献中的标准技术来处理KV中密钥的更新（写入）：将每次访问视为先读后写。

在客户端从与该批对应的服务器接收到B加密值后，它解密、可能更新、然后重新加密这些值并将其发送回服务器。

当一个密钥需要被更新时，客户端会把它添加到UpdateCache中来追踪它是否需要被更新。

## 安全性分析

PANCAKE安全源于以下三点：

1. F作为伪随机函数，E作为（随机化）认证加密方案的密码安全性。这确保了键  $F(k, j)$  看起来是随机的，并且没有任何值泄漏。
2. 假设客户端请求根据  $\pi$  分布，并且我们对  $\pi$  的估计  $\hat{\pi}$  足够好，则每个单独的访问都通过方程1在  $KV'$  上均匀分布。
3. 服务器无法区分假查询和真查询（即，无法推断出任何硬币投掷  $q_{type}$ ）。

## 形式化分析

ROR-CDA: real-versus-random indistinguishability under chosen distribution attack

real world: 对手被给予PANCAKE对KV存储KV'的加密，以及通过在  $\pi$  的  $q$  个样本上运行Batch生成的转录  $\tau$ （其中Batch使用  $\hat{\pi}$ ）。

ideal world: 给对手一个由随机比特串和  $q \cdot B$  均匀随机访问的转录组成的数据库。

实现这一安全目标可以排除基于访问模式泄漏的攻击。

### PANCAKE的ROR-CDA安全理论

Thm: Let  $q \geq 0$  and  $Q = q \cdot B$ . Let  $\pi, \hat{\pi}$  be distributions. For any  $q$ -query ROR-CDA adversary  $\mathcal{A}$  against PANCAKE we give adversaries  $\mathcal{B}, \mathcal{C}, \mathcal{D}$  such that

$$Adv_{PANCAKE}^{ror-cda}(\mathcal{A}) \leq Adv_F^{prf}(\mathcal{B}) + Adv_E^{ror}(\mathcal{C}) + Adv_{Q, \pi, \hat{\pi}}^{dist}(\mathcal{D})$$

where  $F$  and  $E$  are the PRF and AE scheme used by PANCAKE. Adversaries  $\mathcal{B}, \mathcal{C}, \mathcal{D}$  each use  $Q$  queries and run in time that of  $\mathcal{A}$  plus a small overhead linear in  $Q$ .

best case:  $\pi = \hat{\pi}$

## 性能分析

PANCAKE会产生 $B \times$ 的带宽开销，即每批的大小。

$$\Pr [i \text{ queries in queue}] = (1 - \frac{2}{B})(\frac{2}{B})^i$$

$$B \geq 3$$

## 5 处理动态分布

### 5.1 适应分布的变化

#### 副本交换

因为任何分布的副本总数总是 $2n$ ，对于每个获得副本的密钥 $k_i$ ，必须有另一个失去副本的密钥 $k_j$ 。因此，对于所有这些键，处理查询分布中的更改只需要读取 $k_j$ 的值并将其写入 $k_j$ 的副本之一，我们称之为副本交换。

PANCAKE在正常客户端访问之上对副本交换使用捎带确认。

在副本交换期间，修改后的批次使用两个列表： $G$ 和 $L$ 。

$$\begin{aligned} G &= \{(k, j) | k \in S, j \in [R(k, \hat{\pi}, \alpha) + 1, \dots, R(k, \hat{\pi}', \alpha)]\} \\ L &= \{(k, j) | k \in T, j \in [R(k, \hat{\pi}', \alpha) + 1, \dots, R(k, \hat{\pi}, \alpha)]\} \end{aligned}$$

$G$ : 要被添加的副本集

$L$ : 要被删除的副本集

$S$ : 必须获得副本的关键词集

$T$ : 必须获得失去的关键词集

$$R(k, \hat{\pi}, \alpha) = \lceil \hat{\pi}' / \alpha \rceil$$

$|G|$ 始终= $|L|$ ，因为 $|S|=|T|$ 。并且将 $L$ 中的每个副本交换为 $G$ 中的一个副本会导致所有密钥

在 $\hat{\pi}'$ 下具有正确数量的副本。

在回写期间，其值被更新为与G中的副本相关联的值。

PANCAKE在L中的副本标签和将与其交换的G中的副本之间保持映射。

## 调整伪访问分布

我们必须使用不同的伪访问分布来确保对获得副本的密钥的读取始终成功。

$$\text{Temporary} \begin{cases} \alpha \rightarrow \alpha' = \max_k \{ \pi'(k) / R(k, \hat{\pi}, \alpha) \} \\ \tilde{\pi}'_f \text{ to satisfy Equation 1 with } \alpha' \end{cases}$$

对于每个 $(k, j) \in G$ ，我们令 $\tilde{\pi}'_f(k, j) = \frac{\alpha'}{2n\alpha' - 1}$

关键词的现有副本具有 $\tilde{\pi}'_f = \frac{\alpha' - \hat{\pi}'(k) / R(k, \pi, \alpha)}{2n\alpha' - 1}$

对于其他键， $\tilde{\pi}'_f(k, j) = \frac{\alpha' - \hat{\pi}'(k) / R(k, \hat{\pi}', \alpha)}{2n\alpha' - 1}$ 。

## 副本缓存

当检测到分布变化时，PANCAKE会计算每个标签与其对应副本的交换关系。然而，在后续访问中，副本的实际值必须从G传播到L，以确保读取的正确性。如果没有额外的机制，读取G中的副本时可能会获取到错误的值。为确保正确性，在Batch中读取G中的副本时，其值会被缓存到代理中。然后在下次访问时，这个值会被传播到L中的副本，而实际读取则从缓存中提供。

## 插入和删除密钥

尽管我们假设支持大小是固定的，副本交换过程实际上可以支持密钥集合的变化。可以将这视为分布变化 $\text{supp}(\hat{\pi}') \neq \text{supp}(\hat{\pi})$ ，只要PANCAKE初始化时有足够的副本来处理最大支持大小，就可以通过用虚拟副本替换新密钥来插入新键，而删除时则反之操作。虽然需要一些额外的元数据，但类似于PRF标签映射，这些元数据在加密密钥旋转后可以被删除。

## 5.2 安全性分析

ROR-CDDA: “real-or-random security under chosen dynamic distribution attack”它类似于它的静态模拟，除了它使用两个分布 $\pi$ 和 $\pi'$ ：在经过对抗性选择的查询次数后，

分布从  $\pi$  变为  $\pi'$ 。

我们让  $Adv^{ror-cdda}(A)$  成为对手A的ROR-CDDA优势。它捕获了A在分布变化期间区分真实PANCAKE执行 (ROR-CDDA1) 和均匀随机接入 (ROR-CDDA0) 的能力。

经历过多次对抗性选择的查询次数后，分布从  $\pi$  变为  $\pi'$

Thm(simplify):

$$Adv_{PANCAKE}^{ror-cda}(\mathcal{A}) \leq Adv_F^{prf}(\mathcal{B}) + Adv_E^{ror}(\mathcal{C}) + Adv_{Q,\pi,\hat{\pi}}^{dist}(\mathcal{D}_1) + Adv_{Q,\pi',\hat{\pi}'}^{dist}(\mathcal{D}_2)$$

强调一点：ROR-CDDA将从  $\pi$  到  $\pi'$  的转变视为瞬时发生并被检测到，但在某些情况下，这可能不现实，甚至使用PANCAKE中的最先进技术也无法完全保证。因此，PANCAKE可能在检测到变化前就处理来自  $\pi'$  的查询（将其视为来自  $\hat{\pi}$  的样本）。这些查询的分布将是非均匀的，其偏差与  $\pi$  和  $\pi'$  之间的差异有关。如果对手知道这种偏差，利用它进行攻击是可能的，但具有挑战性；实际上，我们并未发现任何已发表的攻击考虑分布变化的可能性。

## 5.3 检测查询分布的变化

结合了两样本Kolmogorov-Smirnov (KS) 检验。回想一下，PANCAKE维护了观察到的访问的直方图H，以维护分布 $\pi$ 的估计值 $\hat{\pi}$ 。PANCAKE通过在代理上维护一个w个最新访问滑动窗口的运行直方图  $H_{running}$  来跟踪分布变化，并使用KS检验判断  $H_{running}$  对应的基础分布是否与  $\hat{\pi}$  不同。如果检验表明发生了变化，PANCAKE会利用当前的  $H_{running}$  快照来更新对新分布 $\pi'$ 的估计  $\hat{\pi}'$ 。

检测分布中的变化并对这些变化做出响应涉及到平衡安全性和效率。test不能太过敏感，也不能太不敏感。

## 6 评价

在各种场景中评估PANCAKE，包括基于主内存和辅助存储的数据存储、静态和动态分布、部署设置和工作负载。

### 比较方法

1. 不安全的基线，不提供安全保证。



这是PANCAKE性能的上限，因为它对应于没有安全开销的数据存储。

## 2. 非递归PathORAM[67] (Z=4)，一种最先进的ORAM。

这是最先进的设计，在我们的模型下（以及在对手可以主动注入自己的查询的更强模型下）提供安全性。

使用两种代表性的存储后端来比较这些方法：内存中的KV存储Redis[57]和基于SSD的持久KV存储RocksDB[59]。对于PathORAM和PANCAKE，客户端查询通过代理服务器转发到数据存储；对于不安全的基线，客户端查询在没有任何中间代理的情况下被转发到后端存储服务器。

## 6.1 静态分布的性能

### 内存服务器存储（Redis，图4）。

PANCAKE的吞吐量显著提高（提高了229倍）。与不安全的基线相比，PANCAKE的平均延迟在2.3-2.6倍之间，吞吐量在6.8-7.6倍之间（图4（a）、4（b））。这是三个因素的累积效应：

1. 由于批量大小 $B=3$ ，带宽开销增加了3倍。
2. 2倍的开销，因为每个请求都会生成一个读写请求
3. 由于加密/解密而产生的开销。

使用多个代理线程，PANCAKE峰值吞吐量在只读工作负载（YCSB工作负载C）的基线的3.4倍以内

对于50%的读、50%的写工作负载（YCSB工作负载A），PANCAKE的吞吐量保持不变，而基线吞吐量增加了约1.8倍，因为基线现在也可以利用全双工链路。

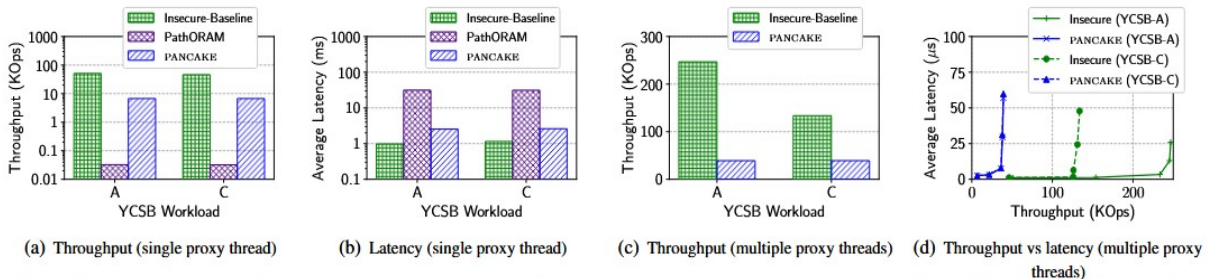


Figure 4: Performance for in-memory server storage (Redis). (a, b) PANCAKE’s throughput is over  $220\times$  higher than PathORAM and within  $\sim 6.8-7.6\times$  of the insecure baseline for a single-threaded proxy; note that the y-axis is in log-scale. (c, d) With multiple proxy threads, PANCAKE’s peak throughput is within  $3.4-6.3\times$  and latency within  $2.3-2.6\times$  of the insecure baseline.

### 基于SSD的服务器存储（RocksDB，图5）。

PANCAKE的表现比PathORAM好17.3倍，与基线相差11.3倍。

对于多个代理线程，PANCAKE峰值吞吐量在只读工作负载的不安全基线的3.3倍以内，在与内存中类似的50%读取、50%写入工作负载的5.3倍以内。

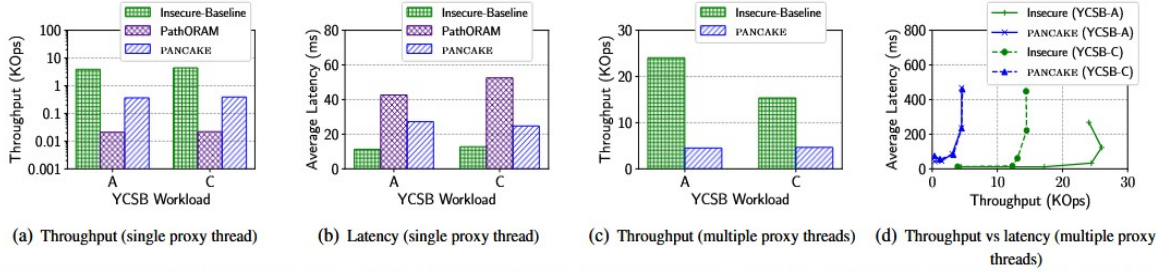


Figure 5: Performance for SSD-based server storage (RocksDB). (a, b) PANCAKE's throughput is  $17.3\times$  higher than PathORAM and within  $\sim 10.7\text{--}11.3\times$  of the insecure baseline for a single-threaded proxy; note that the y-axis is in log scale for (a). (c, d) Using multiple proxy threads, PANCAKE's peak throughput is within  $3.3\text{--}5.3\times$  and average latency within  $2\text{--}2.4\times$  of the insecure baseline.

## 存储开销

PANCAKE的服务器存储需求比我们使用的非递归PathORAM实现低4倍

( $=2 \cdot Z \cdot N$ ,  $Z=4$ )，并且在不安全基线的2倍以内，与两种方法的理论存储开销一致。

PANCAKE的代理存储需求仅占总存储空间的一小部分（约1%），与所有评估工作负载的PathORAM（约0.33%）相似。

## 6.2 适应动态分布

我们使用YCSB Workload-C（只读）评估PANCAKE在隔离状态下（即没有写入影响）检测和适应分布变化的能力。

### 检测分布变化

图6 (a) 测量了Zipf分布的偏度变化；正如预期的那样，该测试在较少的查询中检测到分布的较大变化（例如，偏度从0.99下降到0.0），而不是较小的变化（例如偏度从999下降到0.9）。

图6 (b) 同样，我们观察到，与较小的变化（例如，偏移 $\kappa=1$ ，可能需要数百万次查询）相比，在更少的查询（例如，几十万次）中检测到分布的较大变化（例如偏移 $\kappa=256$ ）。

两种设置的结果都表明，PANCAKE检测分布变化的机制在实践中效果良好，例如，在每秒10万次查询的情况下，PANCAKE可以在1-2秒内检测到变化。

## 适应分布变化

我们评估了PANCAKE在底层分布变化时适应动态分布的开销。特别是，我们将分布从高偏斜（偏斜参数=0.99）变为较小的偏斜，极端情况下为纯均匀访问模式（偏斜参数=0）。

PANCAKE可以在不到25分钟内适应分布的剧烈变化（Zipf到纯均匀）（用于更新所有密钥上新分配的副本），同时在代理服务器上使用<0.05%的服务器存储空间（图6（c））。

这很有趣，有两个原因：

1. PANCAKE观察到，在适应期间，代理存储的增加可以忽略不计
2. 自适应发生在后台，即不停止查询执行，事实上，自适应是在查询执行的基础上进行的。

因此，更高的查询率将导致更快地适应分布的变化。

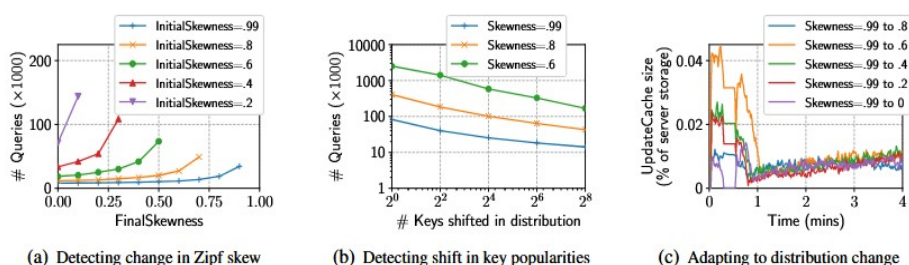


Figure 6: **Handling dynamic distributions.** (a, b) PANCAKE detects larger distribution changes in fewer queries, relative to smaller changes. (c) PANCAKE can adapt from a skewed to uniform distribution with UpdateCache size < .05% of server storage over evaluated workloads.

## 6.3 性能对参数的敏感性

### 代理位置的影响

图7 (a) 使用多个代理线程测量此设置（我们称之为WAN代理）的吞吐量。WAN代理的吞吐量略低于云代理（图4(c)），因为互联网上的可用带宽低于1Gbps，而且往往不稳定。

### 写速率（图7 (b)）和请求分布（图7）对UpdateCache的影响。

我们通过测量UpdateCache的大小来量化其带来的代理存储开销，考虑不同的写入比例和键的底层分布偏斜程度。

图7(b)显示，即使在100%写入的情况下，UpdateCache的大小也始终低于服务器存储的10%。在更现实的低写入率情况下，例如20%写入率时，存储开销降低到服务器存储的

不到3%。

尽管大多数现实世界的分布是高度偏斜的，但在PANCAKE中，具有超过1个副本的关键词的比例在偏斜度降低时会增加。这可能导致UpdateCache大小增加，因为PANCAKE在将写入传播到副本时缓存这些关键词的值。我们通过测量不同偏斜度的YCSB Workload A（50%读-50%写）工作负载的UpdateCache大小来评估这种开销。

图7(c)表明，偏斜度从0.99降低到0.8时，UpdateCache的大小从服务器存储的5%增加到9%，即使在偏斜度较低的情况下，UpdateCache的大小仍然是服务器存储的一个小比例。

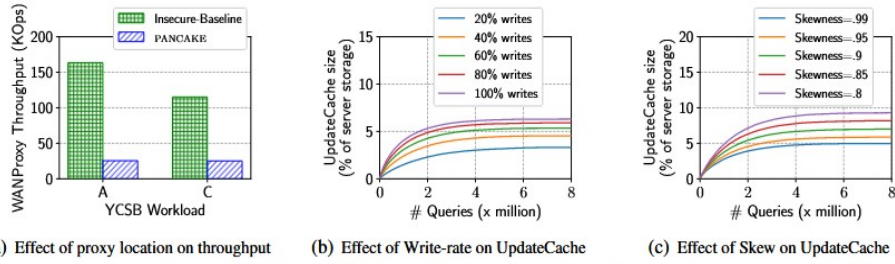


Figure 7: (a) Due to asymmetric and unpredictable available download and upload speeds over the Internet, both the insecure baseline and PANCAKE observe reduced throughput ( $0.65\text{--}0.85\times$ ) for the WAN setting when compared to the cloud setting. (b, c) UpdateCache size increases with write rate for a fixed Zipf distribution (skewness = 0.99) and decreases as skew increases for a fixed write-rate (50%), but remains well below 10% of server storage for all evaluated workloads.

## 批量大小B的影响（图8 (a) - 8 (b)）

根据§4.4，当批处理大小为B时，PANCAKE会产生B倍的带宽开销。图8(a)显示，当网络带宽成为瓶颈时，PANCAKE的吞吐量与B值成正比下降。同时，较大的B值会降低尾延迟，因为请求在查询队列中等待的批次数减少。尽管B=2会导致队列系统不稳定（图8(b)），但B>2几乎没有排队延迟。因此，B在尾延迟和吞吐量之间存在权衡，其中B=3是两者的最佳选择。我们没有评估延迟与批处理大小之间的关系，因为延迟与查询到达间隔时间相关。在固定的到达间隔下，可以从图8(b)推测延迟开销。



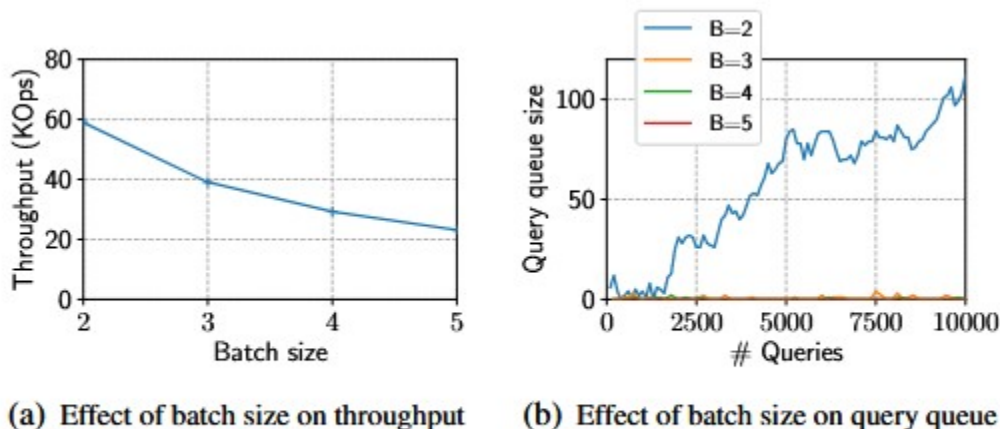


Figure 8: (a, b) Impact of batch size on PANCAKE throughput and query queue size. See §6.3 for discussion.

## 7 讨论

### 查询相关性

目前，PANCAKE假设查询是独立的，但在某些应用场景中查询可能具有相关性。对于包含相关查询的频率分析攻击，尚未有深入研究。作者提出，这需要新的技术工具来应对在现实场景中可能出现的相关访问模式。

### 更强的攻击者

PANCAKE目前的安全模型不包括防止恶意注入查询的攻击（例如回滚攻击），尽管系统使用了认证加密来检测篡改并可通过认证操作计数器来防止回滚。然而，该系统并未设计来抵御能自行发送查询的攻击者。

### 动态分布

PANCAKE的安全性假设分布的变化是瞬时且可即时检测的。尽管评估显示PANCAKE可以在几秒内检测到分布的变化，但若更广泛地支持逐步变化的分布仍有待探索。

### 代理改进

PANCAKE目前使用有状态的代理，存储分布信息、键-副本计数以及待写缓存。未来可以探索更具可扩展性和容错能力的分布式代理实现。

### 改进了代理实现



当前的PANCAKE实现使用了有状态的代理，代理存储了真实和伪造的分布、键-副本计数、待处理的写入请求等。未来可以探索使用分布式代理来提高系统的可扩展性和容错能力。

## 可变大小的值

PANCAKE假设存储的数据值大小固定或被填充到固定的最大长度，以防止基于长度的攻击。然而，这种方法可能在值大小差异大的情况下造成较大的空间开销。未来可以研究如何在不增加存储开销的情况下保护数据不受长度泄露攻击。

## 在基于缓存的系统中隐藏访问模式

在许多基于缓存的系统中（如MySQL结合Memcached），可以通过缓存更频繁访问的键来提升性能，但这会泄露缓存中键的访问频率。为防止访问模式泄露，所有键需以统一频率访问，导致缓存优势消失。初步评估显示，当分布不同或缓存大小受限时，PANCAKE的吞吐量可能较非安全基线下降一个数量级，因此需要新的技术来避免性能下降并同时保护访问模式。

# 8 结论

---

本文在一个新的形式化安全模型中探索了一种新的基于频率平滑的对策，以对抗外包存储的访问模式攻击。我们在一个名为PANCAKE的新系统中实例化了这种方法，该系统是第一个抵抗持久被动对手的访问模式攻击，同时保持存储和带宽的低恒定因子开销的系统。因此，PANCAKE的吞吐量比PathORAM高229倍，在不安全基线的3-6倍范围内。